

Polynomial-Time Reduction and NP-Completeness

Understanding Complexity with AI-Based Modern Enhancement

Team: B.Tech CSE Group • **Subject:** Theory of Computation • **Faculty:** Prof. A. Instructor

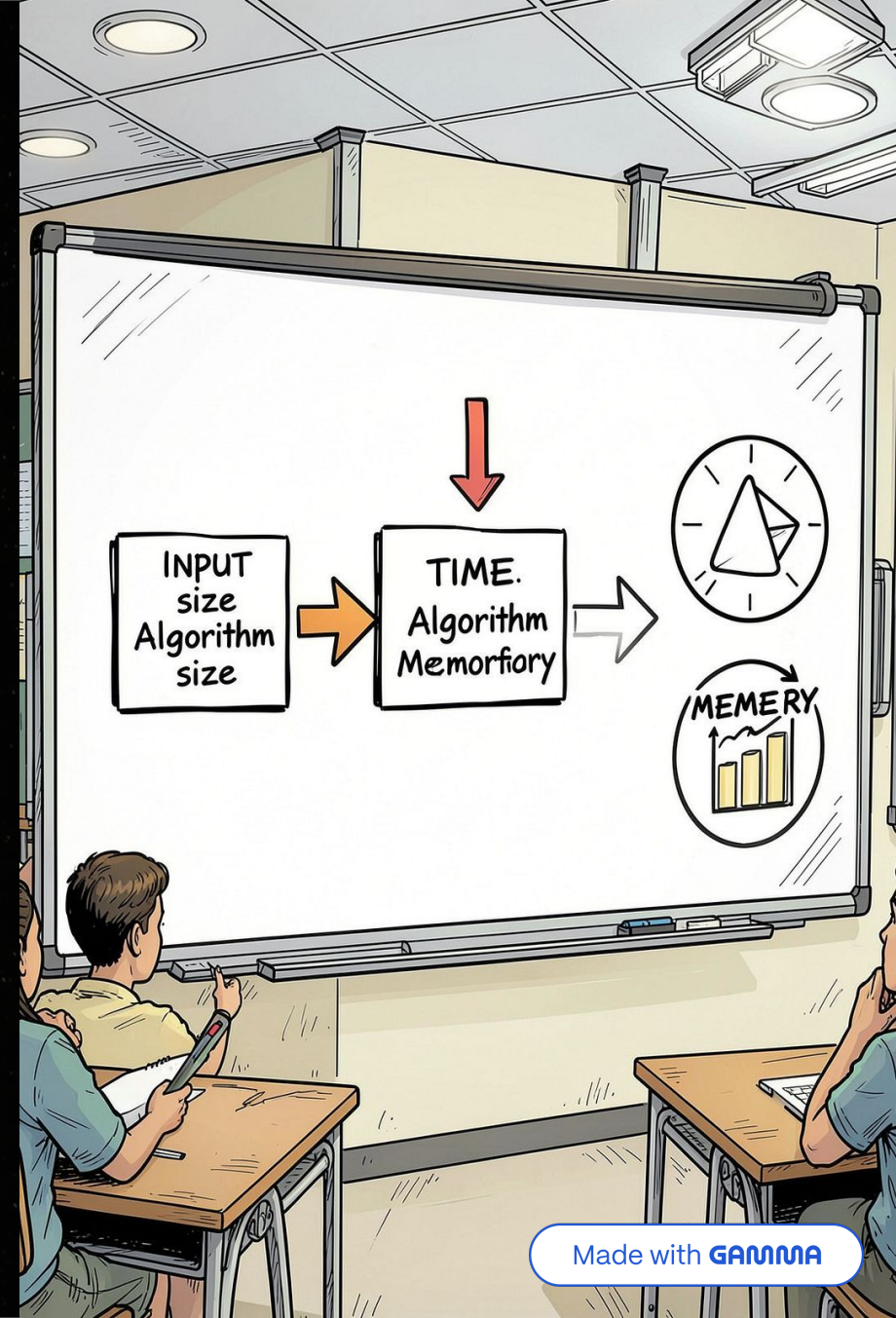
College: Institute of Engineering & Technology



Computational Complexity

Definition: Computational complexity studies the resources required to solve a problem (time, memory).

- Measures problem difficulty
- Depends on input size
- Helps compare algorithms



P and NP

P: Problems solvable in polynomial time.

NP: Problems whose solutions can be verified in polynomial time.



P → Solve Efficiently

Deterministic algorithms with poly-time bound



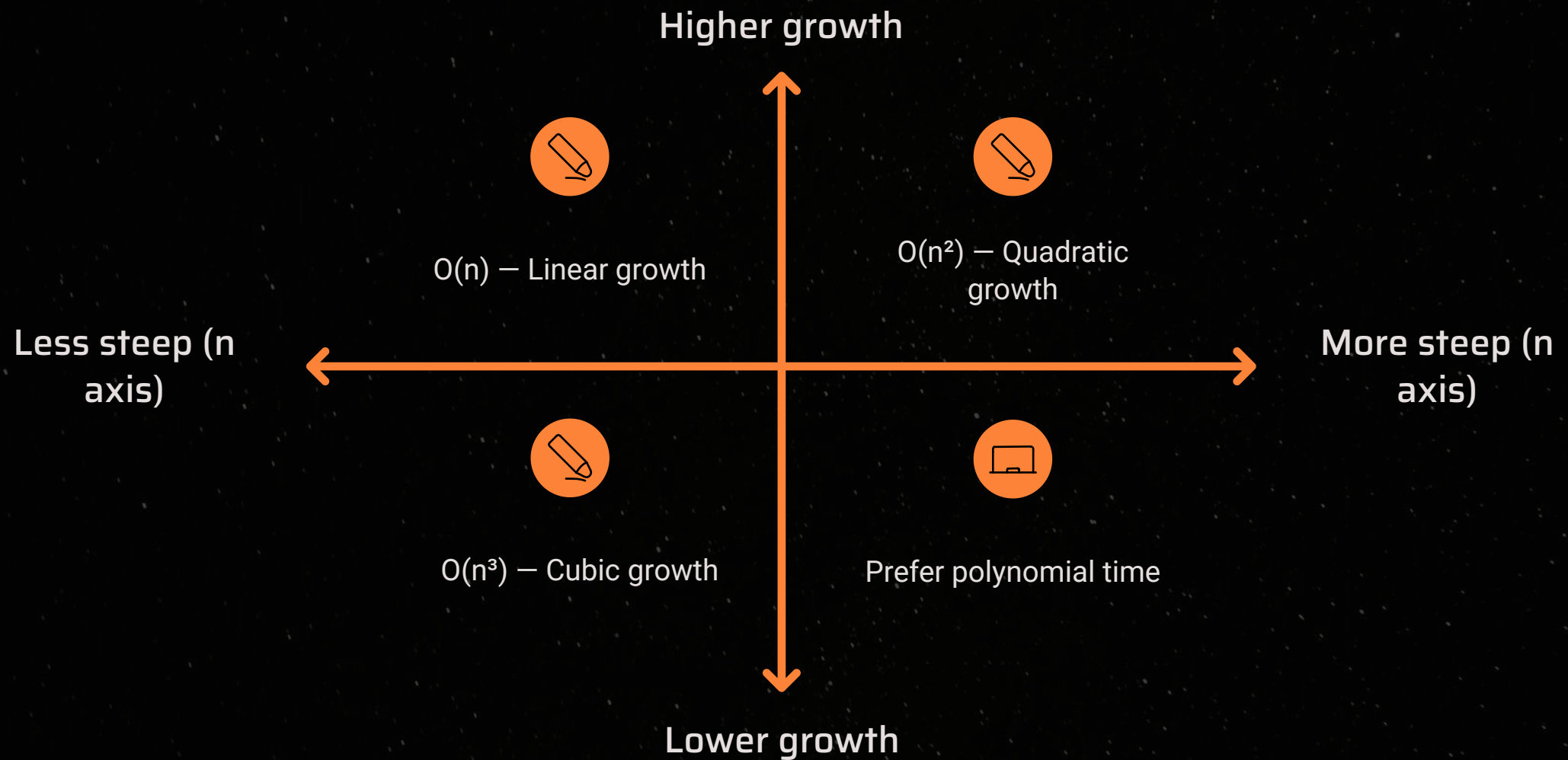
NP → Verify Efficiently

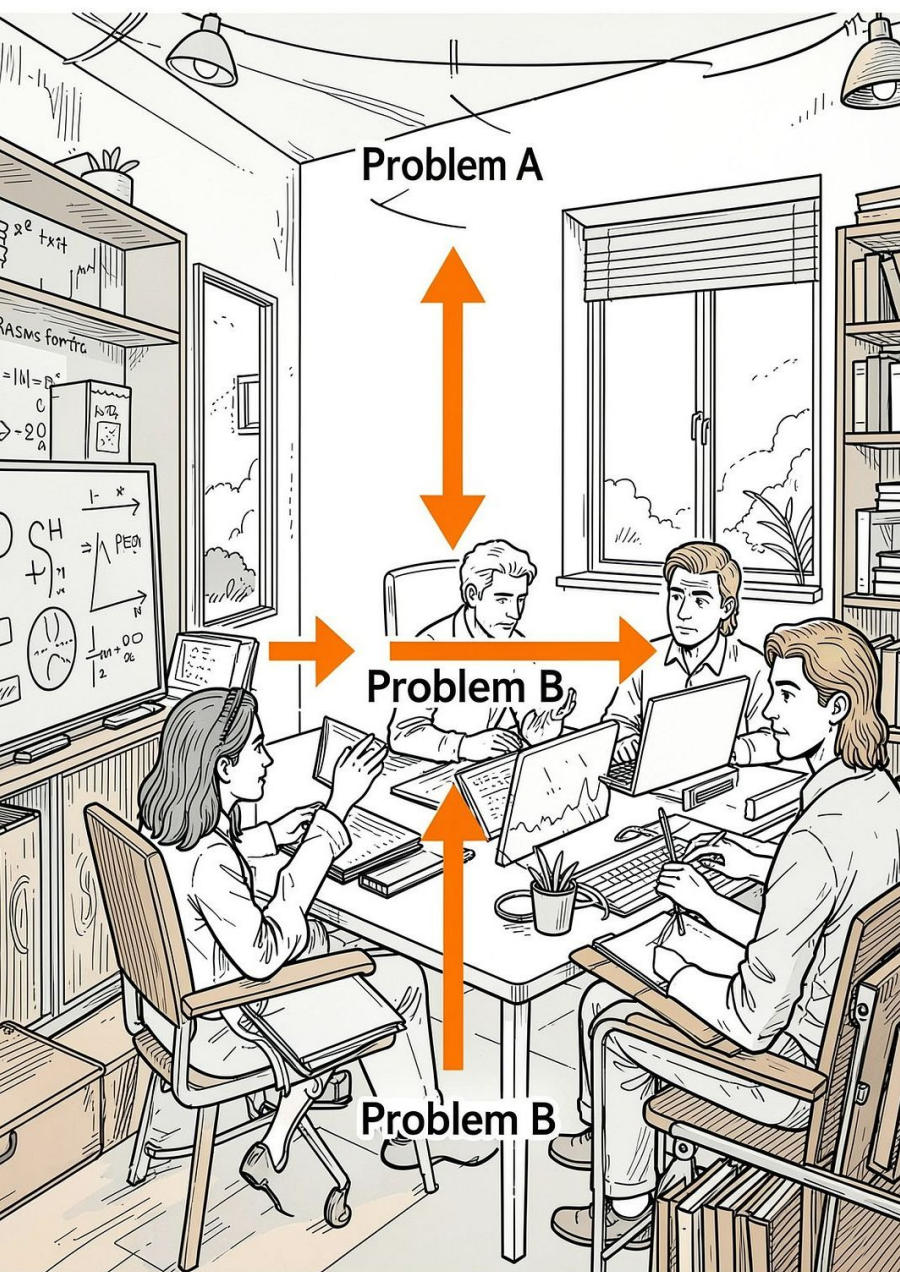
Verification faster than finding solution

Polynomial Time

Definition: Running time expressed as $O(n^k)$, k constant.

- Common forms: $O(n)$, $O(n^2)$, $O(n^3)$
- Preferred for scalable algorithms



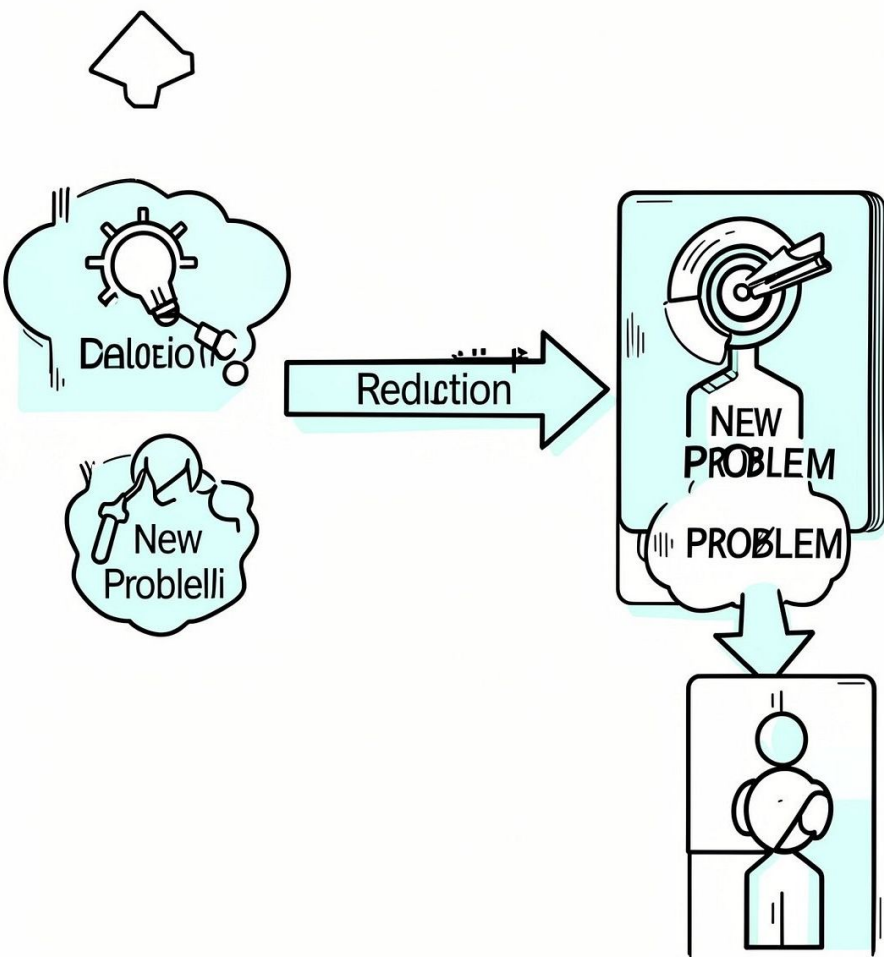


Polynomial-Time Reduction

Definition: Transform one problem to another in polynomial time.

Key idea: Solve B $\rightarrow$ Helps solve A

KNOWN PROBLEM



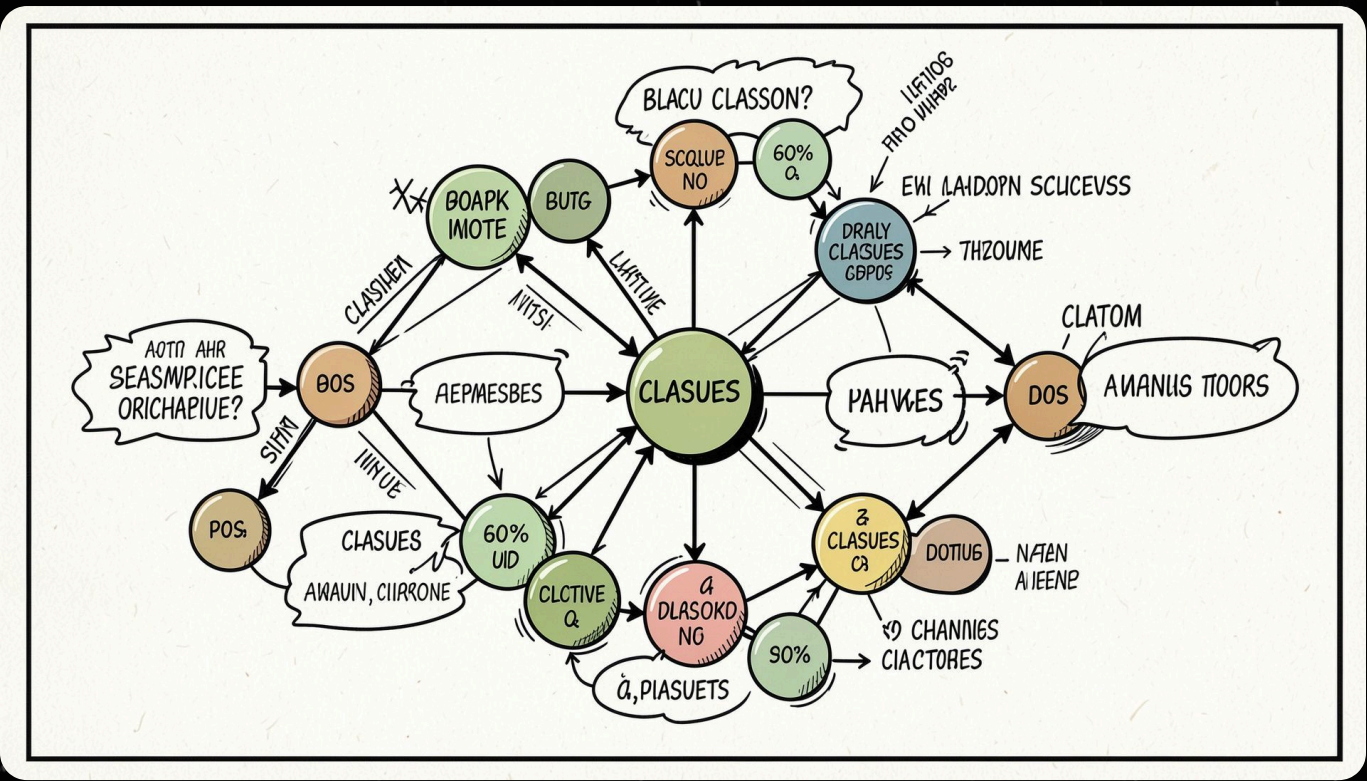
Why Reduction Is Important

- Compares problem difficulty
- Helps prove NP-Completeness
- Transfers known complexity results

Example: 3-SAT → CLIQUE

Concept: 3-SAT can be transformed into a CLIQUE instance in polynomial time.

- Transformation maps clauses to graph structure
- Solving CLIQUE gives a solution for 3-SAT



Visual: Clause groups → nodes;
compatibility → edges; clique corresponds
to satisfying assignment.

NP-Complete

Definition: In NP and NP-Hard.

- Solution can be verified efficiently
- At least as hard as every problem in NP



NP

Efficient verification



NP-Hard

At least as hard



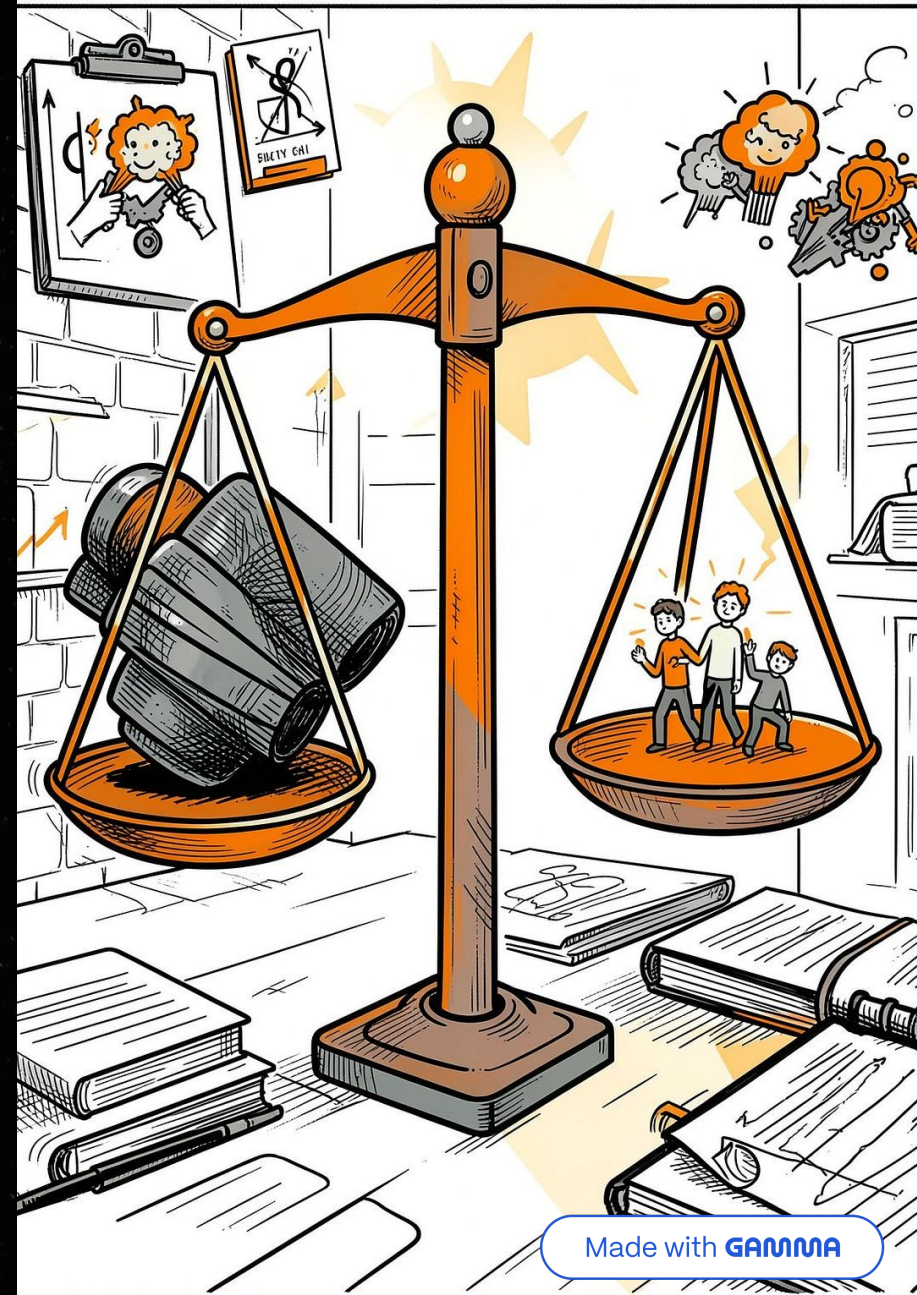
NP-Complete

Both properties hold

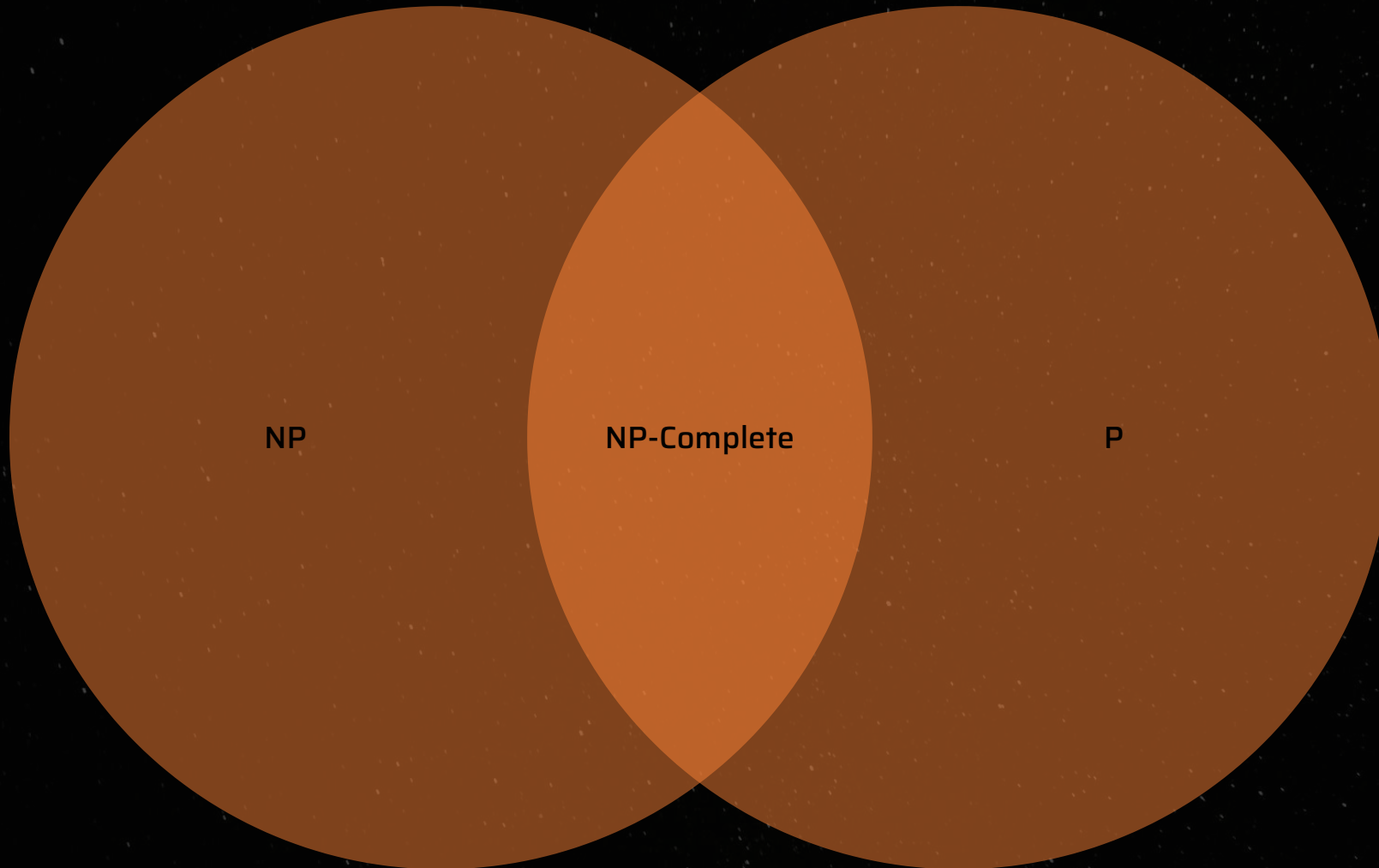
NP-Hard (Short)

Definition: At least as hard as every problem in NP.

- May not be in NP
- Used to show hardness of optimisation problems



P vs NP — Visual Summary



Note: $P = NP$ remains an open problem.